ASSIGNMENT – 11

NAME : SURENDRA REDDY

HALL TICKET NO : 2303A51981

BATCH : 15

Question :

Hyperledger Fabric – Comparative Exploration Objective To study and practically explore Hyperledger
Fabric by setting up a basic Fabric test network, deploying a simple chaincode, and comparing Hyperledger Fabric with a public blockchain (Ethereum) based on architecture, consensus, privacy, performance, and use cases.

Requirements :Software Requirements

Operating System: Ubuntu 20.04 / Windows (with WSL) / macOS • Docker & Docker Compose • Git • cURL • Node.js (v16+) or Go (for chaincode) • Hyperledger Fabric binaries and samples Hardware Requirements • Minimum 8 GB RAM • Internet connection

CODE :

```
import tkinter as tk from tkinter

import ttk, messagebox import

random import time

class  DeFi:          def

__init__(self):

self.balances = {}


   def gas(self):

      return round(random.uniform(0.0001, 0.001), 6)


   def delay(self):

time.sleep(0.3)


   def deposit(self, user, amount):

if amount <= 0:

         return "Invalid amount"
```

```
        self.delay()
gas = self.gas()

        if user not in self.balances:
self.balances[user] = 0

        self.balances[user] += amount        return
f"Deposit Successful\nGas Fee: {gas}"

    def withdraw(self, user, amount):
if user not in self.balances:
return "User not found"

        if self.balances[user] < amount:
return "Insufficient balance"

        self.delay()
gas = self.gas()

        self.balances[user] -= amount        return
f"Withdraw Successful\nGas Fee: {gas}"

    def balance(self, user):
        return self.balances.get(user, 0)
class App:
    def __init__(self, root):        self.defi
= DeFi()        self.root = root
self.root.title("DeFi Tab Interface")
self.root.geometry("500x350")
notebook = ttk.Notebook(root)
```

```
 notebook.pack(expand=True,
 fill="both")

      # Tabs       self.deposit_tab =
tk.Frame(notebook)        self.withdraw_tab
= tk.Frame(notebook)        self.balance_tab
= tk.Frame(notebook)

      notebook.add(self.deposit_tab, text="Deposit")
notebook.add(self.withdraw_tab, text="Withdraw")
notebook.add(self.balance_tab, text="Balance")

      # ------------------ Deposit Tab ------------------
      tk.Label(self.deposit_tab, text="User").pack(pady=5)
self.d_user = tk.Entry(self.deposit_tab)
self.d_user.pack()

      tk.Label(self.deposit_tab, text="Amount").pack(pady=5)
self.d_amount = tk.Entry(self.deposit_tab)
self.d_amount.pack()

      tk.Button(self.deposit_tab, text="Deposit",
command=self.deposit).pack(pady=10)

      self.d_output = tk.Label(self.deposit_tab, text="")
self.d_output.pack()

      # ------------------ Withdraw Tab ------------------
tk.Label(self.withdraw_tab, text="User").pack(pady=5)
self.w_user = tk.Entry(self.withdraw_tab)
self.w_user.pack()
```

```
        tk.Label(self.withdraw_tab, text="Amount").pack(pady=5)
self.w_amount = tk.Entry(self.withdraw_tab)
self.w_amount.pack()

        tk.Button(self.withdraw_tab, text="Withdraw",
command=self.withdraw).pack(pady=10)

        self.w_output = tk.Label(self.withdraw_tab, text="")
self.w_output.pack()

        # ------------------ Balance Tab ------------------
        tk.Label(self.balance_tab, text="User").pack(pady=5)
self.b_user = tk.Entry(self.balance_tab)
self.b_user.pack()

        tk.Button(self.balance_tab, text="Check Balance",
command=self.check_balance).pack(pady=10)

        self.b_output = tk.Label(self.balance_tab, text="")
self.b_output.pack()

    def deposit(self):
        try:
            user = self.d_user.get()            amount
= float(self.d_amount.get())          result =
self.defi.deposit(user, amount)
self.d_output.config(text=result)       except:
            messagebox.showerror("Error", "Invalid input")
    def withdraw(self):
        try:
```

user = self.w_user.get() amount = float(self.w_amount.get()) result = self.defi.withdraw(user, amount)

self.w_output.config(text=result) except:

messagebox.showerror("Error", "Invalid input")

def check_balance(self): user = self.b_user.get()

balance = self.defi.balance(user)

self.b_output.config(text=f"Balance: {balance} ETH")

if __name__ == "__main__":

root = tk.Tk()

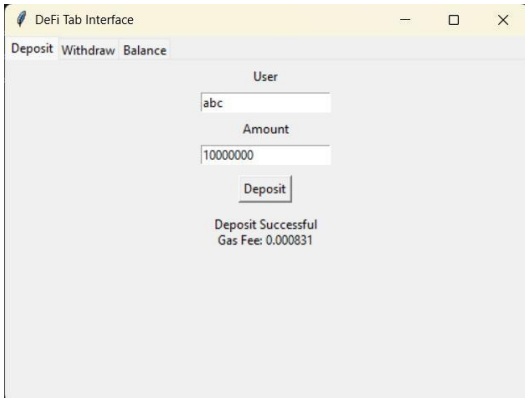
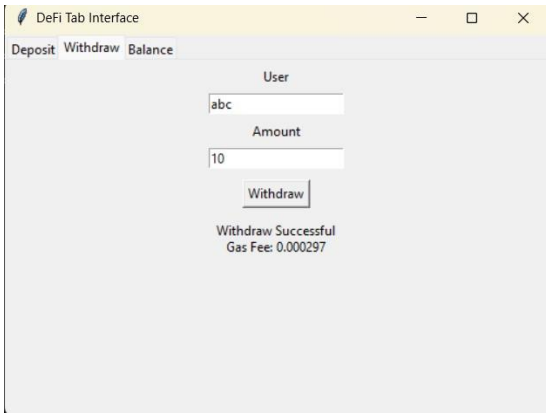
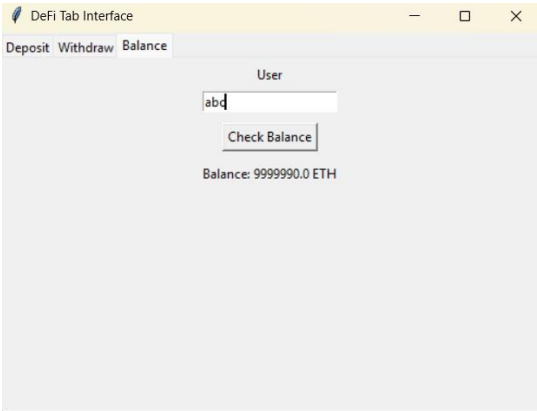
app = App(root)

root.mainloop()

IMPLEMENTATION :

C: > Users > Rushi > Downloads > gui.py > ...

```python
import tkinter as tk
from tkinter import ttk, messagebox
import random
import time
class DeFi:{} time
    def __i{} timeit
        sel{} timer
            {} _testimportmultiple
    def gas(self):
        return round(random.uniform(0.0001, 0.001), 6)

    def delay(self):
        time.sleep(0.3)

    def deposit(self, user, amount):
        if amount <= 0:
            return "Invalid amount"

        self.delay()
        gas = self.gas()

        if user not in self.balances:
            self.balances[user] = 0

        self.balances[user] += amount
        return f"Deposit Successful\nGas Fee: {gas}"

    def withdraw(self, user, amount):
        if user not in self.balances:
            return "User not found"

        if self.balances[user] < amount:
            return "Insufficient balance"
```

```
C: > Users > Rushi > Downloads > gui.py > ...
class App:
    def __init__(self, root):
                    command=self.check_balance).pack(pady=10)

        self.b_output = tk.Label(self.balance_tab, text="")
        self.b_output.pack()

    def deposit(self):
        try:
            user = self.d_user.get()
            amount = float(self.d_amount.get())
            result = self.defi.deposit(user, amount)
            self.d_output.config(text=result)
        except:
            messagebox.showerror("Error", "Invalid input")

    def withdraw(self):
        try:
            user = self.w_user.get()
            amount = float(self.w_amount.get())
            result = self.defi.withdraw(user, amount)
            self.w_output.config(text=result)
        except:
            messagebox.showerror("Error", "Invalid input")
    def check_balance(self):
        user = self.b_user.get()
        balance = self.defi.balance(user)
        self.b_output.config(text=f"Balance: {balance} ETH")
if __name__ == "__main__":
    root = tk.Tk()
    app = App(root)
    root.mainloop()
```

OUTPUT :